Assignment 2
Computer Vision

Student Name: Shreyash Narayane
Student ID: 25934391

Date: 22-Apr-2026

94691 - Deep Learning
Master of Data Science and Innovation
University of Technology of Sydney

Table of Contents

1. Introduction 2

2. Data Understanding 3

3. Part A: Architecture 4

Model 1 - ResNet 4

Model 2 - GoogLeNet 5

Model 3 - MobileNetV3 5

4. Transfer Learning Training Process 6

Experimental Setup 6

Training Configuration 6

5. Part A - Results and Analysis 7

6. Part A - Limitations 11

7. Model selection for Part B 11

8. Part B - Fine Tuning Approach 12

Setup 12

Experiment 1 (Unfreeze Layer 4 Only) 12

Experiment 2 (Unfreeze Layer 3 & 4) 13

9. Part B - Results and Analysis 14

10. Limitations and Recommendations 17

Limitations 17

Recommendations 17

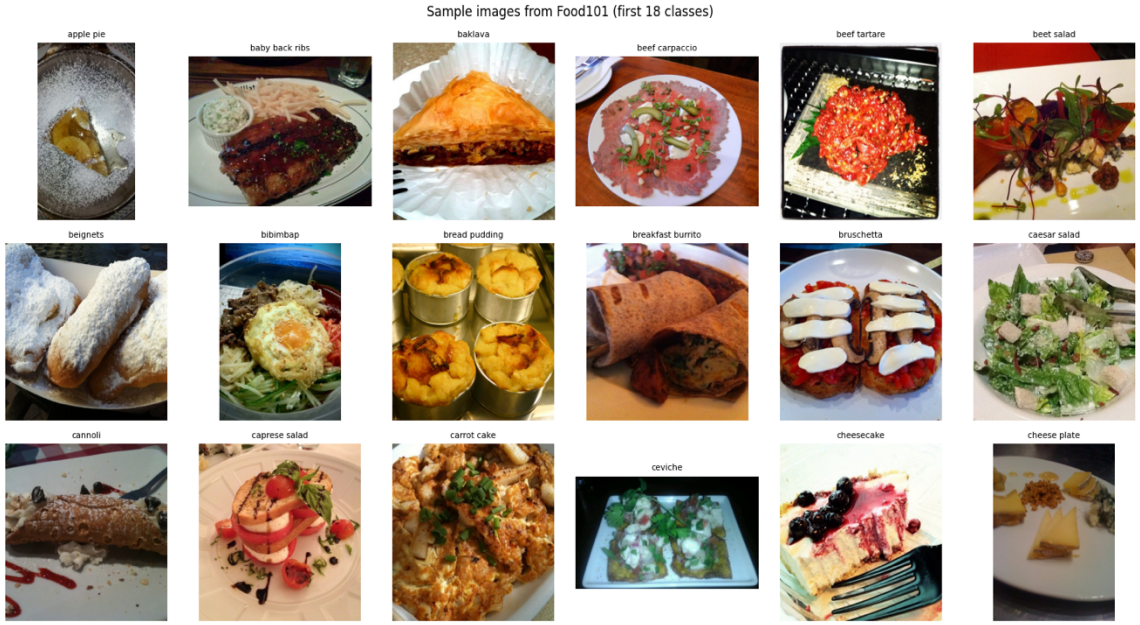
11. Conclusion 18

12. References 18

1. Introduction

This report covers the work done in Assignment 2, which has two parts. Part A involved taking three pre-trained convolutional neural networks (CNNs), replacing their classification heads, and training those heads on the Food101 dataset while keeping the backbone frozen. The three architectures used were ResNet-50, GoogLeNet, and MobileNetV3-Small. Part B extended the best-performing model from Part A using fine-tuning, where selected layers of the backbone were gradually unfrozen and retrained on the same dataset.

The goal was to understand how much of the classification performance comes from the frozen ImageNet features alone, and how much can be gained by allowing parts of the backbone to adapt to the target domain. All training was done using PyTorch on MPS device, and results were tracked epoch-by-epoch with validation accuracy as the selection criterion for the best model state.



2. Data Understanding

The Food101 dataset contains 101,000 images spread across 101 food categories. Each category has exactly 750 training images and 250 test images, making it a perfectly balanced dataset with no class imbalance concerns. Images vary in resolution and aspect ratio but were originally rescaled so the maximum side length does not exceed 512 pixels.

For this assignment, 15% of the official training set was held out as a validation split, using a fixed random seed (42) to ensure reproducibility. The remaining 85% was used for training. This gave the following splits:

Split	Samples	Per Class
Train (after Val split)	64,388	~637
Validation (15% of train)	11,362	~113
Test (official)	25,250	250

All images were resized to 224×224 pixels during preprocessing to match the input dimensions expected by ImageNet-pretrained backbones. Training images were augmented with random horizontal flipping and random rotation up to ±10 degrees. Normalisation used ImageNet channel statistics (mean: [0.485, 0.456, 0.406], std: [0.229, 0.224, 0.225]) for both training and test sets. No augmentation was applied at validation or test time.

The dataset is visually demanding. Many classes share similar textures and colours, for example different preparations of steak, various noodle dishes, or dumplings. This inter-class similarity is the main difficulty the models face.

3. Part A: Architecture

Three pre-trained CNN backbones were used. Each was originally trained on ImageNet, a 1000-class image recognition dataset with roughly 1.2 million images. The key architectural differences between them directly affect how well their frozen features transfer to a 101-class food classification task.

Model	Params (M)	Feature Dim	Year	Key Innovation
ResNet-50	25.6	2048	2015	Residual connections
GoogLeNet	6.6	1024	2014	Inception modules
MobileNetV3-Small	2.5	576	2019	Depthwise-sep + SE attention

Model 1 - ResNet

ResNet-50 was introduced by He et al. in 2015 and solved the degradation problem that prevented very deep networks from training effectively. The central idea is the residual (skip) connection, instead of learning a direct mapping from input to output, each block learns a residual function added back to the input. This means gradients have a direct path through the skip connection, which prevents vanishing gradients in deep architectures.

ResNet-50 uses a bottleneck design in each residual block, with three convolution layers (1×1, 3×3, 1×1) that compress and then re-expand the channel dimensions. This lets the network go to 50 layers without the explosion in compute that would come from using only 3×3 convolutions throughout. The final global average pooling layer produces a 2048-dimensional feature vector before the classification head.

The large feature vector (2048 dimensions) is an advantage in transfer learning; more dimensions mean more room for the classification head to find discriminative directions in feature space.

Model 2 - GoogLeNet

GoogLeNet (Inception v1) was introduced by Szegedy et al. in 2014. Its defining contribution was the Inception module, which applies multiple convolution filter sizes (1×1, 3×3, 5×5) in parallel within the same layer and concatenates their outputs. The reasoning was that different feature scales are useful at different depths, and letting the network learn which ones matter, rather than committing to a single filter size, should produce richer representations.

During original training, GoogLeNet also used two auxiliary classifiers at intermediate layers to inject gradients into the earlier parts of the network. In transfer learning (Part A), only the final output is used, so those auxiliary paths are inactive. This means the frozen features at GoogLeNet's final pooling layer may not be as sharp as those of ResNet, because the intermediate supervision that helped shape them during pre-training is absent from the inference path.

GoogLeNet produces a 1024-dimensional feature vector, which is smaller than ResNet-50 and may limit its contribution to its weaker transfer performance in this setup.

Model 3 - MobileNetV3

MobileNetV3-Small was introduced by Howard et al. in 2019, with the explicit goal of high accuracy at very low parameter counts and compute budgets. It uses depthwise-separable convolutions, which factorizes a standard convolution into a depthwise step (one filter per channel) and a pointwise step (1×1 convolution across channels). This dramatically reduces the number of multiply-accumulate operations compared to standard convolutions.

MobileNetV3 also incorporates Squeeze-and-Excitation (SE) attention blocks. Each SE block computes a global average per channel, passes it through two small fully connected layers, and produces a set of per-channel weights. These weights scale the feature maps, so the network learns to suppress uninformative channels and emphasize informative ones. The result is that MobileNetV3's 576-dimensional feature vector is compact but more selectively informative than a naive 576-dim vector would be.

This architecture was primarily designed for mobile and embedded deployment, optimized for efficiency rather, but its SE mechanism gives it an edge over GoogLeNet in frozen-backbone transfer despite the much smaller feature dimensionality.

4. Transfer Learning Training Process

Experimental Setup

The transfer learning setup was identical across all three models. Each backbone was loaded with ImageNet pre-trained weights, all backbone parameters were frozen using requires_grad = False, and the original classification head was replaced with a new three-layer fully connected head. The same head architecture was used for all three models:

- Linear(in_features → 512) → BatchNorm1d(512) → ReLU → Dropout(0.3)
- Linear(512 → 256) → BatchNorm1d(256) → ReLU → Dropout(0.3)
- Linear(256 → 101) - output layer for 101 food classes

Input dimensions per model: ResNet-50: in_features = 2048 | GoogLeNet: in_features = 1024 | MobileNetV3-Small: in_features = 576

Training Configuration

All three models were trained with identical hyperparameters to allow fair comparison:

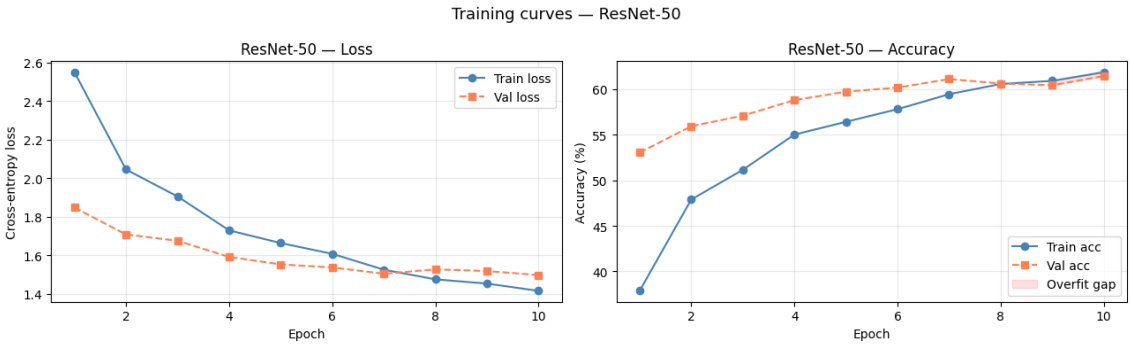
- **Loss function**: CrossEntropyLoss - standard choice for multi-class classification where classes are mutually exclusive
- **Optimizer**: Adam with learning rate 1e-3 - adaptive learning rates per parameter make it faster to converge when only the head is being trained
- **Scheduler**: StepLR with step_size=3 and gamma=0.5 - halves the learning rate every 3 epochs, allowing stable convergence in later epochs without oscillating around a minimum
- **Epochs**: 10
- **Batch size**: 32 - balances memory usage and gradient stability
- Only the classification head parameters were passed to the optimizer; backbone parameters were excluded entirely using filter(lambda p: p.requires_grad, model.parameters())

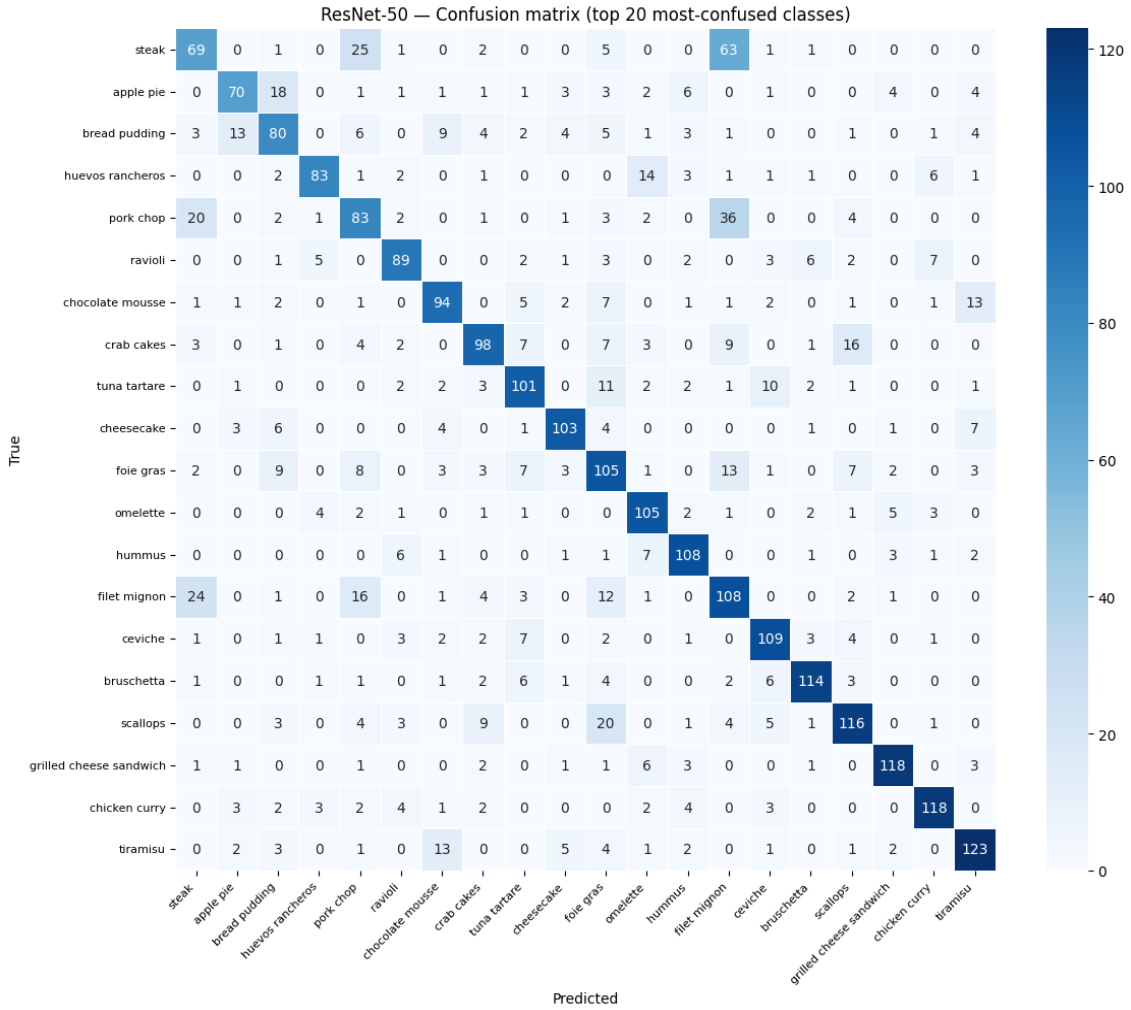
BatchNorm layers in the head serve two purposes: they normalize the input to each linear layer, which speeds up training and reduces sensitivity to weight initialization, and they introduce a mild regularization effect. Dropout(0.3) further reduces overfitting by randomly zeroing 30% of neuron activations during training. These two mechanisms together are the only regularization applied to the head, since the frozen backbone cannot contribute to or benefit from any regularization applied upstream.

The best model state (based on validation accuracy) was saved at the end of each epoch and restored at the end of training. This means the returned model carries the weights from whichever epoch generalized best, not necessarily the final epoch.

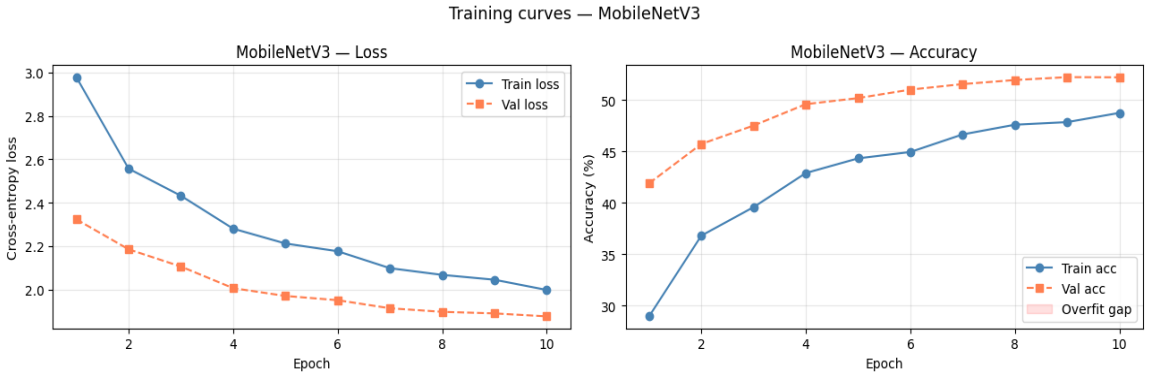
5. Part A - Results and Analysis

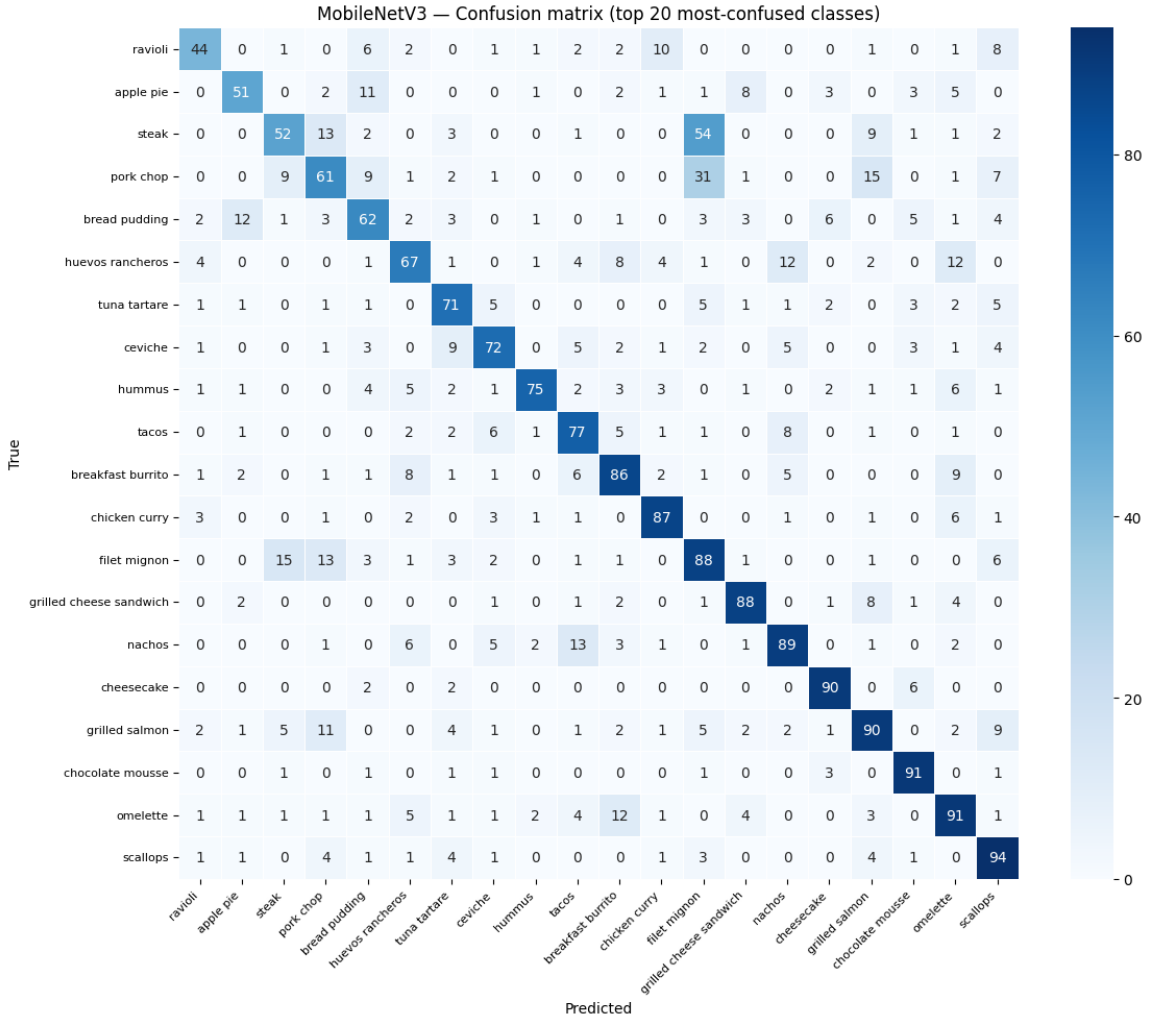
ResNet-50: The 2048-dimensional feature vector gives the head significantly more information to work with than either of the other two architectures. Even with no backbone training, the combination of residual features and a reasonably sized head is enough to correctly classify two out of every three test images. The training curves showed a clear but moderate overfitting gap in the later epochs, expected when only the head is trainable.



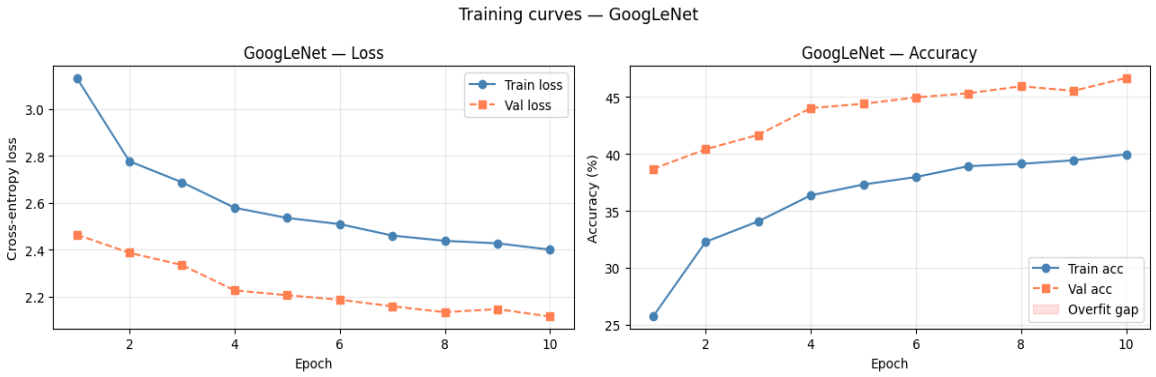


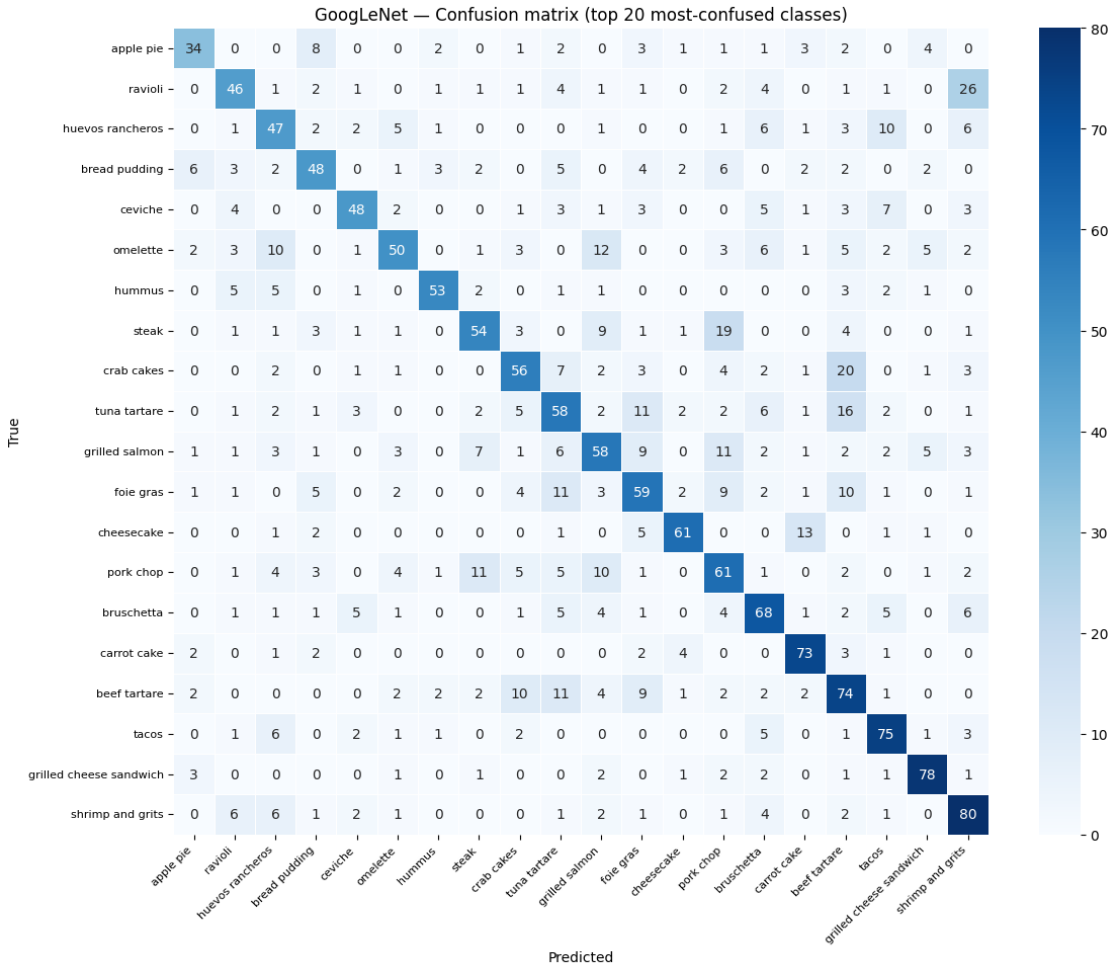
MobileNetV3-Small: Despite having the smallest feature vector (576 dimensions), it outperformed GoogLeNet by roughly 5.5 percentage points. This confirms that the SE attention mechanism in MobileNetV3's backbone produces more task-relevant features per dimension. The model trains fast (fewer parameters) and converges quickly, but the tight 576-dim bottleneck limits how much information the head can extract.





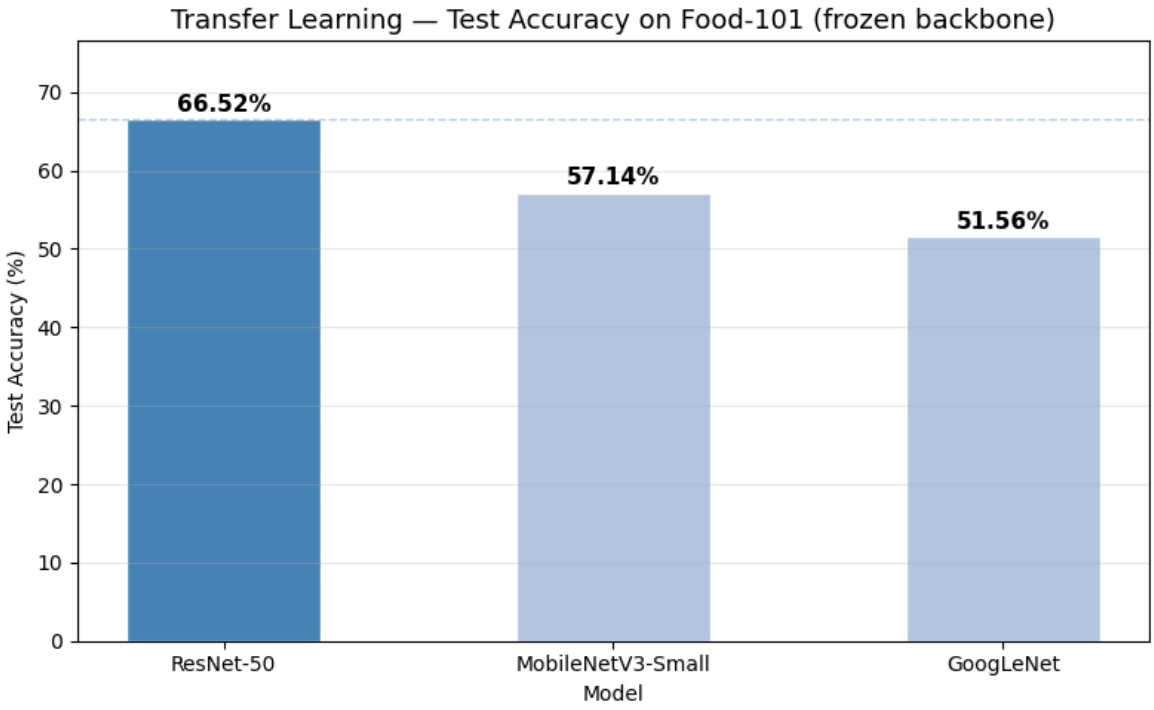
GoogLeNet: The weakest result at 51.56%. The 1024-dim feature vector is larger than MobileNetV3's but produces worse results, suggesting that raw dimensionality is not the deciding factor. The absence of auxiliary classifier supervision during this evaluation path is a likely contributing factor: those paths were active during ImageNet training and influenced the feature representations, but they are not active in standard inference.





Model	Feature Dim	Trainable Params	Test Accuracy (%)
ResNet-50	2048	~1.21M	66.52
MobileNetV3-Small	576	~0.72M	57.14
GoogLeNet	1024	~0.95M	51.56

Across all three models, the confusion matrices show a consistent failure pattern: errors cluster around visually similar fine-grained food categories. Different steak preparations, noodle variants, and dumpling types are frequently confused. These categories share textures and colors that are difficult to separate using ImageNet features alone, since ImageNet was never optimized for intra-category food distinctions.



ResNet-50 achieved the highest test accuracy at 66.52%, followed by MobileNetV3-Small at 57.14% and GoogLeNet at 51.56%.

6. Part A - Limitations

The most fundamental limitation is the frozen backbone itself. All three models are constrained to produce features that were optimized for broad ImageNet object recognition, not for distinguishing between 101 food categories. Fine-grained visual differences, especially in texture-heavy categories like different meat preparations or noodle types, are not necessarily captured in these fixed representations.

The training ran for only 10 epochs with a relatively simple augmentation strategy (horizontal flip and ±10° rotation). More aggressive augmentation, such as color jitter, random cropping, or cutmix, could potentially push validation accuracy higher while the backbone remains frozen. However, the ceiling imposed by the fixed features limits how much augmentation alone can help.

GoogLeNet may benefit from more training time. Its accuracy was still improving slowly at epoch 10, and it likely needs more epochs to converge due to the auxiliary paths not being present during fine-tuning mode.

7. Model selection for Part B

ResNet-50 was selected for fine-tuning in Part B. Three reasons drove this decision:

- Highest Part A test accuracy (66.52%) - it provides the strongest frozen baseline to build on
- Residual connections make it safer to unfreeze layers: gradients always have a direct skip path, which reduces the risk of catastrophic forgetting when backbone weights are updated
- The clear layer structure (conv1 → layer1 → layer2 → layer3 → layer4 → fc) makes it straightforward to selectively unfreeze specific depth levels, enabling a controlled comparison between shallow and deeper unfreezing

8. Part B - Fine Tuning Approach

Setup

The Part A checkpoint for ResNet-50 (best validation accuracy weights) was loaded as the starting point for both experiments. The backbone was restored in its fully frozen state, and each experiment selectively unfroze different layer groups. Both experiments were run independently from the same checkpoint to ensure that the results are comparable and not influenced by the training path of the other experiment.

Key changes from Part A training:

- Optimizer: differential learning rates were used - a lower backbone learning rate to update pre-trained weights gently, and a higher head learning rate to continue adapting the classification head
- Scheduler: **CosineAnnealingLR** (T_max = 15, eta_min = 1e-6) replaced StepLR. Cosine decay avoids the abrupt LR drops of StepLR that can destabilize partially unfrozen backbone weights
- Epochs: 15 (increased from 10, giving newly unfrozen layers more time to adapt)

Experiment 1 (Unfreeze Layer 4 Only)

Layer4 is the deepest residual block group in ResNet-50, immediately before the global average pooling and classification head. It produces the most task-specific, high-level features. Unfreezing only layer4 is a conservative first step: it exposes roughly 14.96M backbone parameters to gradient updates, while the remaining backbone layers (conv1, layer1, layer2, layer3) stay frozen and continue to produce their general-purpose low-level and mid-level features.

Differential LRs: layer4 at 1e-4, classification head at 1e-3. The 10× lower rate for the backbone prevents the pre-trained weights from changing too aggressively and losing the useful representations learned on ImageNet.

Total trainable parameters: 16,172,645 (65.4% of the model). The remaining 34.6% stays frozen.

Experiment 2 (Unfreeze Layer 3 & 4)

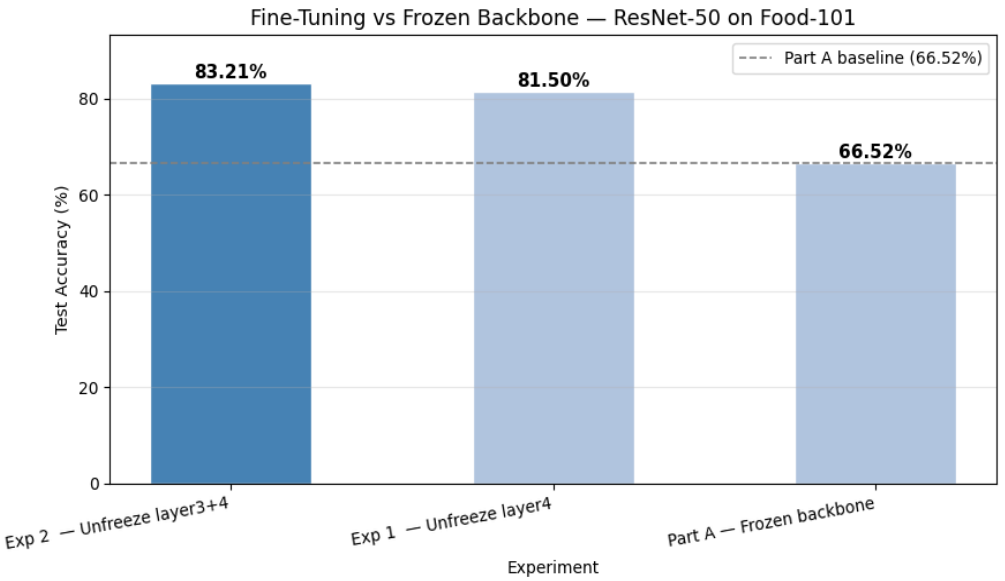
Experiment 2 unfrozen both layer3 and layer4, adding approximately 7M more trainable backbone parameters on top of Experiment 1. Layer3 contains mid-level feature detectors that are somewhat more specific to the domain than layer1 or layer2, but more general than layer4. Allowing it to adapt means the model can start reshaping the mid-level representations that layer4 then processes.

Differential LRs: backbone at 5e-5 (halved compared to Experiment 1), head at 1e-3. The more conservative backbone rate compensates for the increased number of backbone parameters being updated - with 22M backbone params now in play, a higher LR would risk disrupting the mid-level features that still need to be somewhat general.

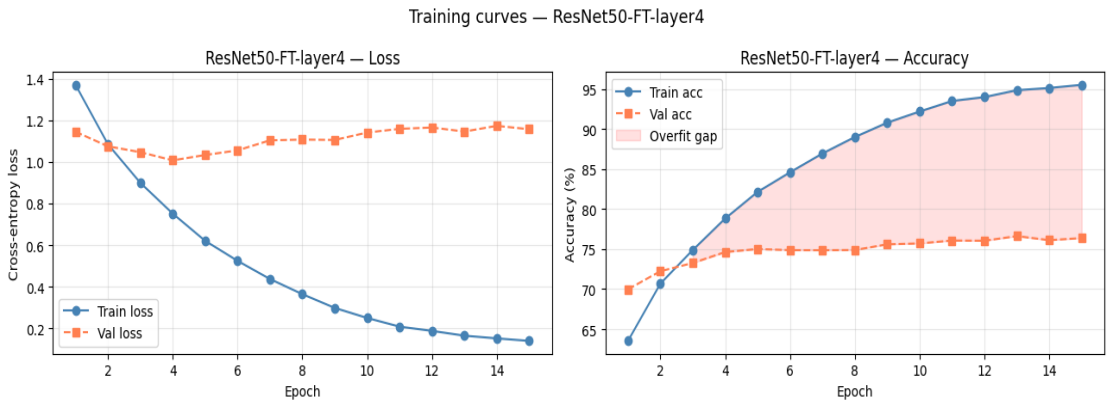
Total trainable parameters: 23,271,013 (94.2% of the model). Only conv1, bn1, layer1, and layer2 remain frozen, preserving the low-level edge and texture detectors that transfer well across most visual domains.

9. Part B - Results and Analysis

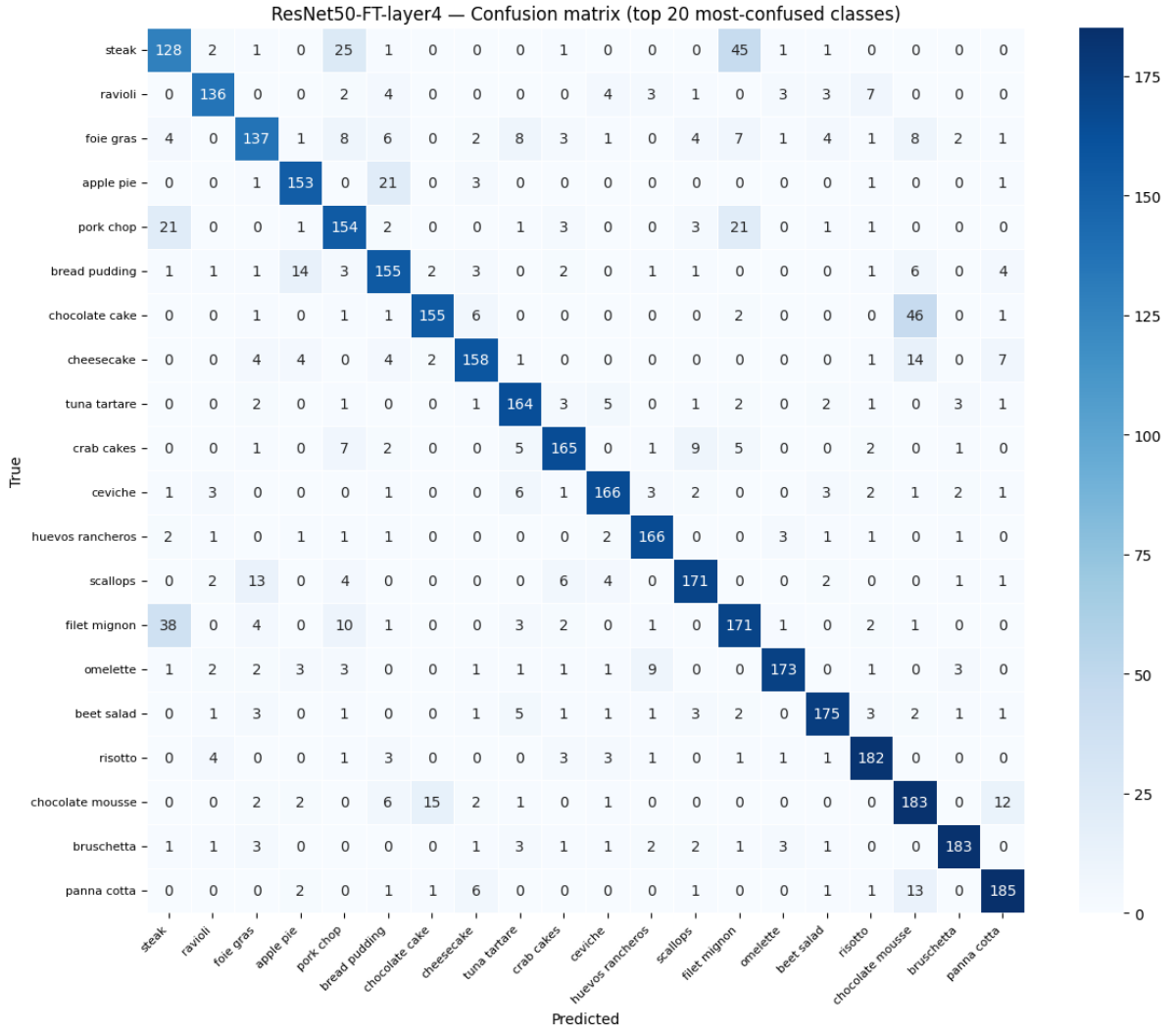
Experiment	Unfrozen Layers	Backbone LR	Head LR	Epochs	Test Acc (%)
Part A - Frozen backbone	None	—	1.00E-03	10	66.52
Exp 1 - Unfreeze layer 4	Layer 4	1.00E-04	1.00E-03	15	81.5
Exp 2 - Unfreeze layer 3 + 4	**Layer 3 + layer 4**	5.00E-05	1.00E-03	15	**83.21 (Best)**



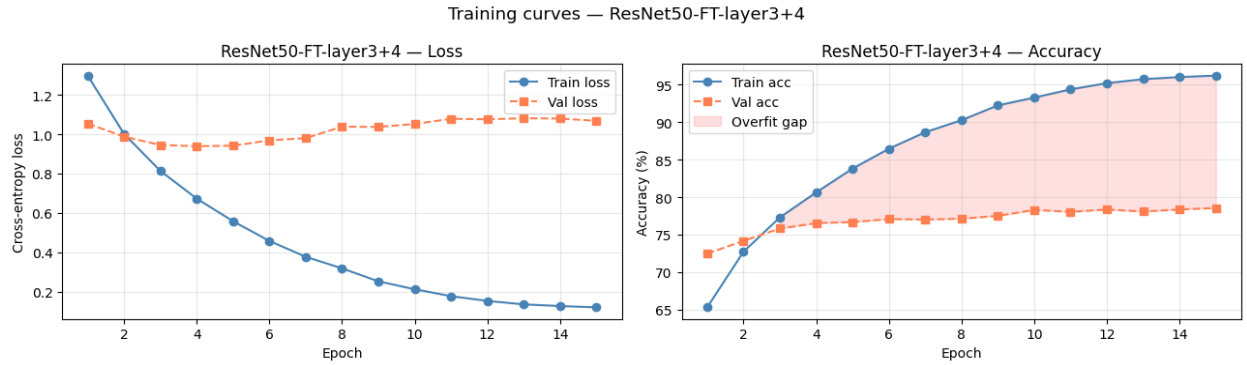
Experiment 1 (layer4 unfrozen) - 81.50% test accuracy, Macro F1: 0.815: A 14.98 percentage point improvement over the Part A baseline of 66.52%. Validation accuracy climbed steadily from 69.98% at epoch 1 to a peak of 76.63% at epoch 13. The fact that it reached this peak at epoch 13 rather than epoch 15 suggests the model found a good generalization point and then started to drift slightly, which is why best-weight restoration matters, the test evaluation uses the epoch 13 checkpoint.



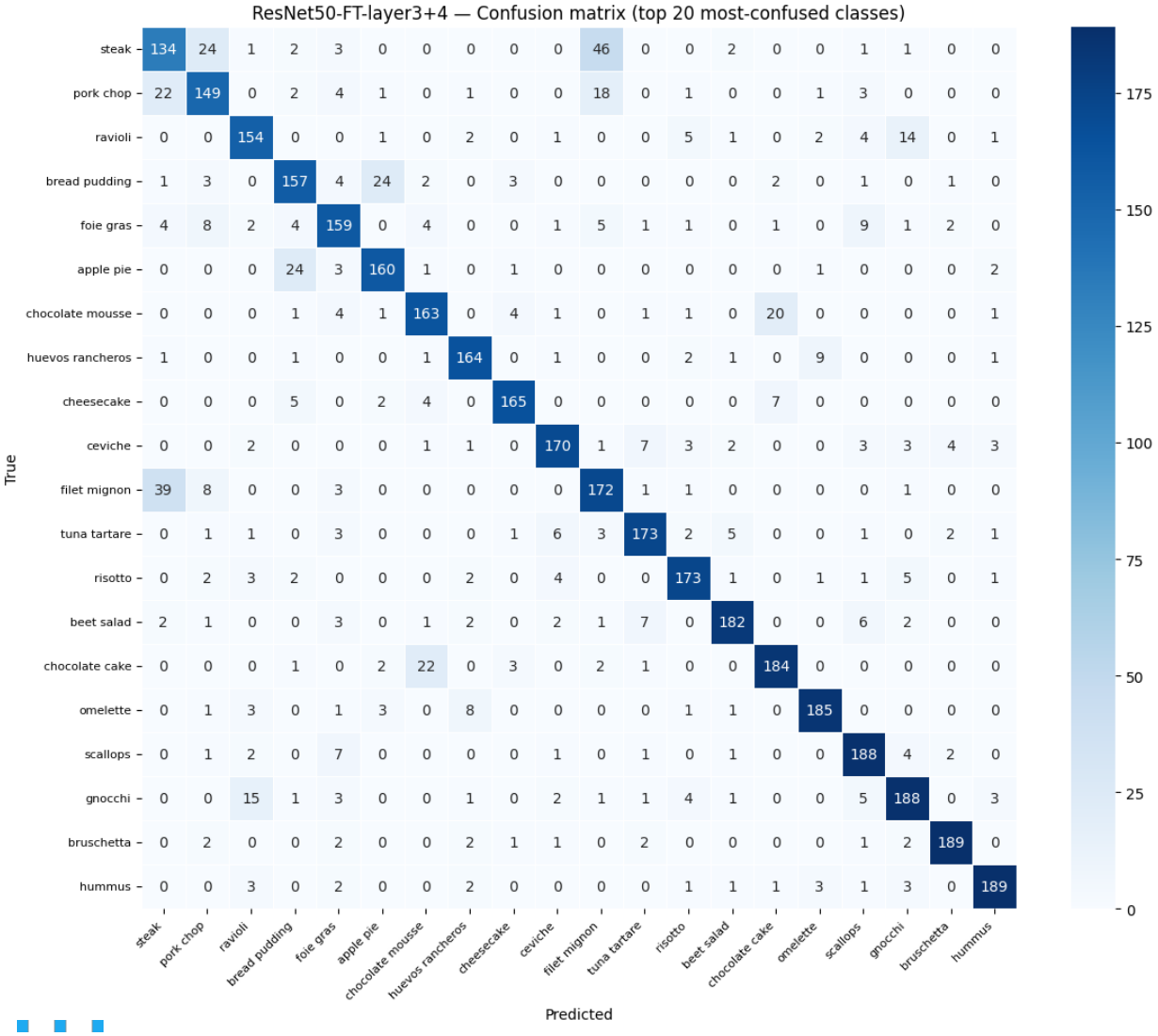
The training accuracy reached 95.55% by epoch 15, meaning the widening train/val gap in the later epochs reflects moderate overfitting. The cosine decay scheduler reduces this risk compared to StepLR, but the gap still opens because there are far more trainable parameters than in Part A and the dataset size relative to those parameters decreases.



Experiment 2 (layer3+4 unfrozen) - 83.21% test accuracy, Macro F1: 0.832: A 16.69 percentage point improvement over Part A and a 1.71-point gain over Experiment 1. What is notable here is that Experiment 2's validation accuracy was still climbing at epoch 15 - it did not plateau as clearly as Experiment 1, suggesting the model had not fully converged and additional epochs could push accuracy higher. Despite nearly 7M more trainable parameters than Experiment 1, the train/val gap in Experiment 2 is comparable, not larger. This confirms that the halved backbone learning rate (5e-5) is doing its job: the additional parameters are adapting in a controlled way rather than overfitting aggressively.



Looking at the per-class results, the hardest classes across both experiments remain the same visually similar meat and dumpling categories. Experiment 2 shows a modest but consistent improvement at the bottom of the F1 distribution compared to Experiment 1, steak F1 rises from 0.516 to 0.536, pork_chop from 0.580 to 0.589. The easiest classes (edamame: F1 = 0.992, macarons: 0.952) remain near-perfect in both experiments. The validation accuracy overlay chart (both experiments plotted against the Part A baseline) shows that Experiment 2 tracks above Experiment 1 from epoch 1 onward. Neither experiment dips below the Part A baseline at any epoch, confirming that fine-tuning consistently improves on the frozen backbone setup.



10. Limitations and Recommendations

Limitations

Both fine-tuning experiments still show a meaningful train/val accuracy gap by the end of training, indicating overfitting. The gap is not catastrophic, but it does mean the model is memorizing some training-specific patterns rather than generalizing perfectly. The relatively simple data augmentation (flip and ±10° rotation) likely contributes to this.

Experiment 2 had not converged by epoch 15, validation accuracy was still rising at the final epoch. This means the best achievable accuracy from the layer3+4 unfreezing strategy has not yet been reached with the current training budget.

The hardest food categories were different steak preparations, pork chop variants, and noodle types remain the primary failure mode. These categories require the model to detect very fine-grained texture and color differences that even the fine-tuned backbone struggles with. Two unfreezing strategies were tested (layer4 only, and layer3+4). Deeper unfreezing strategies (including layer2 or unfreezing from the top down) could be explored.

Recommendations

- Run Experiment 2 for more epochs (20 to 25). Since the validation accuracy was still rising at epoch 15, additional training would likely push accuracy above 83.21% without changing any other hyperparameter.
- Apply stronger data augmentation: color jitter, random grayscale, and random erasing would make the training distribution more varied and help close the train/val gap.
- Try progressive unfreezing: start with layer4 only, then unfreeze layer3 after a few epochs, then layer2. This staged approach gives each newly unfrozen group time to warm up before more layers are added.
- Experiment with a larger image size (e.g. 256×256 or 320×320) during fine-tuning. Larger inputs preserve more spatial detail, which could help with the fine-grained categories where texture is the key discriminator.
- Consider test-time augmentation (TTA): averaging predictions over multiple augmented views of each test image at inference time typically gives a 1-2% accuracy boost without any additional training.

11. Conclusion

This assignment covered transfer learning and fine-tuning on the Food101 dataset using three CNN architectures. In Part A, ResNet-50 achieved the best frozen-backbone accuracy at 66.52%, outperforming MobileNetV3-Small (57.14%) and GoogLeNet (51.56%). The size and quality of the feature vector each backbone produces are the primary factors that determine frozen transfer performance.

In Part B, fine-tuning ResNet-50 with two different unfreezing strategies both delivered substantial improvements over the Part A baseline. Unfreezing layer4 alone raised test accuracy to 81.50%, and further unfreezing layer3 alongside layer4 pushed it to 83.21% the best result across all conditions. The key to making deeper unfreezing work without degrading performance was using a conservative backbone learning rate and cosine annealing to keep gradient updates gradual.

The results confirm that frozen-backbone transfer provides a reasonable starting point for a task like food classification, but fine-tuning is essential to reach competitive accuracy. The deeper the unfreezing and the more carefully the learning rates are calibrated, the more the model adapts to the target domain without losing the general representations from ImageNet pre-training.

12. References

- **He, K., Zhang, X., Ren, S., & Sun, J. (2016).** Deep residual learning for image recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 770–778. https://doi.org/10.1109/CVPR.2016.90 (Original ResNet publication)
- **Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., & Rabinovich, A. (2015).** Going deeper with convolutions. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 1–9. https://doi.org/10.1109/CVPR.2015.7298594 (GoogLeNet/Inception publication)
- **Howard, A., Sandler, M., Chu, G., Chen, L.-C., Chen, B., Tan, M., Wang, W., Zhu, Y., Pang, R., Vasudevan, V., Le, Q. V., & Adam, H. (2019).** Searching for MobileNetV3. Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 1314–1324. https://doi.org/10.1109/ICCV.2019.00140 (MobileNetV3 publication)
- Generative AI tools (ChatGPT, Claude, and Copilot) were utilized to assist in drafting, structuring, and ideation for this report.
- PyTorch Documentation, torch.nn module: https://pytorch.org/docs/stable/nn.html